

HRU — Status Report and Technical Assessment

Oct 1, 2026 · @OMEN

The harness is complete and producing results, but HRU's core mechanism has not yet been tested, and the durability metric needs repair before any claim is final.

Project: ForgetBench · HRU (Hybrid Retention-aware Unlearning)

Team

Name	Register no.	Programme	Email
Bulla Joel Arpith	25WU0103011		
Kanchusthambham Jayanth	25WU0103021	B.Tech, BIC	jayanth.k_2029@woxsen.edu.in
Macha Sai Kamal	25WU0103026	B.Tech CSE, BIC	saikamal.m_2029@woxsen.edu.in
Shaik Shahul Parvez	25WU0103036	B.Tech, BIC	shahul.shaik_2029@woxsen.edu.in
Damera Sai Sri Ranga Charan	25WU0102048	B.Tech CSE (AI & ML)	charan.damera_2029@woxsen.edu.in

Mentor: Prof. Geetha Tripathi, Woxsen University

1. Executive summary

One finding is solid. The headline durability result (HRU -1.309 vs full retraining -0.582) is **not yet evidence that HRU works**, for three reasons.

- 1. **HRU's defining mechanism was never in the pipeline that produced the numbers.**
The attack-then-reweight loop from the proposal exists in the July codebase (`arms/hru.py` , `_AttackHardener`). The Colab notebooks used a probe that only decided when to stop. It never changed what the model learned.
- 2. **The durability metric mostly measures damage.** Across all twelve arms, durability correlates with retain-set degradation at $r = +0.88$. Naive gradient ascent, the baseline expected to fail, scores 83% of HRU's durability at one third of the cost. Adjusted for damage, HRU still ranks second only to full retraining.

3. The winning arm differs from NPO in four ways at once: three times the scrub budget, sharpness-aware optimisation, a restricted parameter set, and no sharding. The gain cannot be attributed to any one of them.

What is solid: SISA-style sharding with logit averaging dilutes approximate unlearning. A scrubbed shard alone reaches -4.179 ; inside a four-shard ensemble it reads -0.502 , and durability falls from -1.309 to -0.420 .

Next: repair the metric, port the real HRU mechanism, and run an equal-budget comparison over three seeds. That is about 30–40 minutes of T4 GPU time.

2. Corrections to earlier claims

Four statements in the progress slide (slide 11), its speaker notes and presentation script v2 are not supported by the data. Claude wrote them during the analysis; this section replaces them.

Earlier claim	Why it does not hold	Use instead
"The ablation proves the gain comes from the hardening loop"	The compared arms differed in sharding <i>and</i> hardening. Holding sharding fixed: scrub-only -0.459 vs HRU -0.420 , so no hardening effect. The probe never fed back into the loss.	"The clear effect is that logit-averaged sharding dilutes approximate unlearning. Inline hardening has not been tested yet."
"HRU is $2.2\times$ more durable than full retraining"	The metric tracks model damage ($r = +0.88$). A ratio of log-probabilities is also not a meaningful "times" figure.	"Unsharded HRU shows the lowest post-attack forget likelihood (-1.309 vs -0.582). We are checking whether that is fact-specific."
"HRU's MIA of 0.433 is closest to the ideal 0.5"	Retraining saw neither the forget nor the holdout set, yet scores 0.371. The sets differ in distribution, so 0.371 is the real baseline.	Report MIA relative to 0.371. SISA (0.381) and RMU (0.412) sit closer than HRU.
Slide 11 row label "HRU inline-hardened"	That arm contains no working inline hardening.	"HRU (unsharded, SAM scrub)"

3. What has been built

Five Colab notebooks run end to end and save everything to Google Drive (`MyDrive/HRU_project`). Two follow-up notebooks are built but not yet run.

Notebook	Purpose	Status
01 Data + base model	Synthetic corpus (130 fictional profiles: 20 forget, 80 retain, 30 holdout), GPT-2 fine-tune, sanity checks	Run, passed
02 Harness + reference	Full metric suite on the model before any unlearning	Run
03 Arms A and B	Full retrain, SISA, sharded control, gradient ascent, NPO, RMU	Run
04 Arm C (v3)	HRU with SAM scrub, config search, ablation, shard-count sweep	Run
05 Benchmark	Recovery curves, cost vs durability, amortised cost, sequential deletions	Run
04b Aggregation sweep	Weighted vs veto shard aggregation; no training	Built, not run
06 HRU v4	Cost-optimised variant: no shards, plateau stopping, alternating SAM	Built, not run

Two codebases have diverged. The July harness in the project folder (`harness.py` , `arms/` , `eval/`) has never produced results. The Colab notebooks produced every number here but lack the July `_AttackHardener` .

Metrics recorded for every arm:

Metric	What it measures	Better is
Forget log-prob	Mean log-probability of correct answers on the forget set	Lower
Retain log-prob	Same, on the retain set	Higher
Truth ratio	$P(\text{wrong answers}) \div P(\text{correct answer})$	Near 1.0, like a model that never saw the data
MIA AUC	Min-K% membership inference, forget vs holdout	Near retrain's 0.371

Metric	What it measures	Better is
Durability	Forget log-prob after a 15-step relearning attack on unseen paraphrases	Lower
Cost	Optimizer updates per deletion request	Lower

Safeguards already in place: a dev/test split of attack probes, a sharded control arm, answer-only loss masking, global seeding, and comma-safe results logging.

4. Method: HRU as designed vs as measured

The measured HRU differs from the proposed HRU at the one step that makes it novel.

Stage	As designed (proposal, <code>arms/hru.py</code>)	As measured (Colab v2/v3)	v4 (built, not run)
Localize	Replay only retain shards least entangled with forget authors	SISA data split; shard logits averaged	No shards
Scrub	NPO loss against a frozen reference	NPO with SAM, mid-stack MLP weights only (22.8%)	Same, SAM on alternate steps
Harden	Every 5 steps, attack a clone for 3 SGD steps; a batch whose loss drops over 30% is replayed at 2× weight	Attack a copy after each round; leakage used only to decide when to stop	No attack probes; stop when sharpness plateaus
Effect on learning	Model is trained against what relearns easily	None, because the stop threshold was never reached	None

When early stopping never fires, the measured "hardening loop" is a plain SAM scrub. The only difference is that each probe resets the random seed, which reshuffles batches. That is why sharded HRU (−0.420) and scrub-only (−0.459) are indistinguishable.

v4 cuts projected cost from 240 to 40–120 updates, but it optimises a method whose value is unproven. Without attack-then-reweight it is essentially SAM-based NPO, close to arXiv:2502.05374. Its cost techniques belong after the mechanism is validated.

5. Experimental setup

Item	Value
Base model	GPT-2 (124M parameters), fine-tuned 6 epochs on forget + retain
Forget set	20 fictional profiles × 5 facts = 100 question-answer pairs
Retain set	80 profiles = 400 pairs
Holdout set	30 profiles = 150 pairs, never trained on (MIA non-members)
Relearning attack	15 fine-tune steps at learning rate 5e-5 on paraphrased forget facts; 40 dev and 40 test probes
Hardware	Google Colab, one T4 GPU
Seeds	1 (rows marked seeds = 2 mix two separate runs; see §7.4)
Cost unit	Optimizer updates; one NPO scrub pass = 40 updates = 1×

6. Results

Unsharded HRU has the lowest post-attack forget likelihood (−1.309); full retraining keeps the best utility (retain −0.236). Single seed, notebook 05.

Arm	Forget LP	Retain LP	Truth ratio	MIA	Durability	Updates
A_retrain	−0.259	−0.236	1.007	0.371	−0.582	150
A_shard_control	−0.388	−0.390	0.994	0.391	−0.380	0
A_sisa	−0.478	−0.459	1.023	0.381	−0.457	26
B_ga	−4.100	−1.421	1.665	0.314	−1.088	40
B_npo	−4.871	−1.594	13.166	0.275	−0.762	40
B_rmu	−0.567	−0.578	0.986	0.412	−0.450	40
C_scrub_only (sharded)	−0.604	−0.484	1.155	0.363	−0.459	120
C_hru_full (sharded)	−0.502	−0.444	1.059	0.358	−0.420	120
C_harden_noshard	−4.179	−1.692	1.292	0.433	−1.309	120
C_hru_k2	−0.701	−0.415	1.096	0.416	−0.378	120

Arm	Forget LP	Retain LP	Truth ratio	MIA	Durability	Updates
C_hru_k4	-0.532	-0.460	1.088	0.362	-0.439	120
C_hru_k8	-0.702	-0.694	1.029	0.361	-0.670	120

HRU updates count SAM optimizer updates (120). The notebook printed 360 "steps" because SAM uses two passes per update and diagnostic probes added 120 more; neither changed what the model learned.

Cost. Against one NPO scrub pass: SISA 0.65×, full retraining 3.75×, every HRU variant 3.0× (6.0× with probes). The design target was 2–5×. At this toy scale retraining is artificially cheap; in deployment it is closer to 100× a scrub pass.

7. Findings

7.1 Logit-averaged sharding dilutes approximate unlearning (solid)

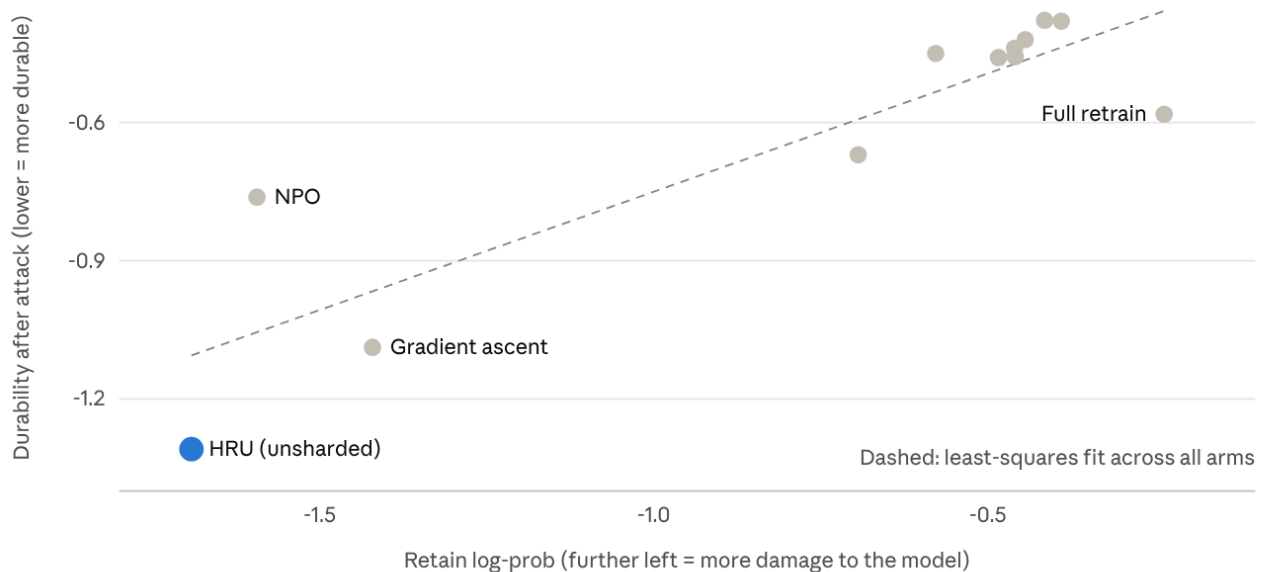
Three untouched shards outvote the one scrubbed shard 3-to-1. Scrubbed shard 0 alone reaches a forget log-prob of -4.179. Inside the four-shard ensemble the same shard reads -0.502, and durability is -0.420 against -1.309 for the same procedure without sharding.

SISA's averaging is right for exact unlearning, where the retrained shard has nothing to suppress. It fails for approximate unlearning, where the scrubbed shard carries a signal the others cannot express. Notebook 04b tests weighted and veto (minimum) aggregation. The unsharded figure averages two runs, so the size of the effect is approximate; its direction, on a threefold gap, is not.

7.2 The durability metric is confounded with model damage (critical)

The most durable arms are the most damaged ones

Correlation $r = +0.88$ across 12 arms



Notebook 05 results · 12 arms · single seed

The three most durable arms are the three that damaged the model most. Three observations point to the cause:

- **Template tokens dominate the metric.** It averages the whole answer ("Soren Solis was born in Calderfen."), where only one token is the fact. The base model scores the forget set at -0.210 and never-seen profiles at -0.271 , a gap of just 0.06 nats, though it generates the correct facts.
- **Retraining looks as if it knows the forget set.** It never saw a forget fact, yet scores -0.259 , higher than the sharded control (-0.388), which did. It simply learned the answer template better.
- **The attack lowers the score of models that never unlearned.** The reference model falls from about -0.21 to -0.57 over 40 attack steps. A real relearning attack should raise it; the attack data's different template ("X grew up in ...") pulls probability away from the original wording.

The encouraging part. Measured against the fit line, unsharded HRU is more durable than its damage predicts by 0.20, second only to full retraining (0.22). NPO is the worst outlier, 0.29 less durable than its damage predicts. So HRU's lead is not only damage, but the size of any real effect is unknown until the metric is fixed.

Until fact tokens are scored on their own, every durability comparison in this report is preliminary.

7.3 Truth ratio: the one encouraging fact-specific signal

Truth ratio compares the correct answer with wrong answers that differ only in the fact token, so templates do not dominate it. A model that has truly forgotten is indifferent (about 1.0), as full retraining shows. Among the aggressive scrubbers, unsharded HRU is closest to that (1.29), against gradient ascent (1.67) and NPO (13.17). NPO now prefers wrong answers, a sign of over-forgetting. These are pre-attack values; measuring truth ratio after the attack is the most informative addition available.

7.4 Data hygiene

- One seed: no result here is statistically significant.
- Mixed rows: C_harden_noshard, C_hru_full and C_scrub_only each average a v2 run (plain NPO) with a v3 run (SAM). Wipe and regenerate `results.csv` before quoting any number outside the team.
- MIA's effective baseline is 0.371, not 0.5 (§2).

7.5 Sequential deletions

Over three successive deletion requests, retain log-prob moved $-1.95 \rightarrow -2.22 \rightarrow -1.82$ and general perplexity $205 \rightarrow 193 \rightarrow 225$. Three noisy points show no trend. The reference model's perplexity is not in these results, so the level of fluency damage needs that baseline before it can be read.

8. Limitations

1. The core HRU mechanism is untested (§4).
2. The durability metric is confounded with model damage (§7.2).
3. One seed only, and some rows mix pipeline versions (§7.4).
4. Toy scale: GPT-2 124M and 100 forget facts. Short templated answers make the metric confound worse than it would be on free-form data such as TOFU or MUSE.
5. Forget and holdout sets differ in distribution, which biases MIA (§2).
6. The prior-art check is incomplete. SAM for relearning-resilient unlearning (arXiv:2502.05374) and meta-learned tamper resistance sit close to HRU. Novelty must be argued narrowly: first-order replay of attack-identified hard examples inside the scrub.

9. Plan to a defensible result

Steps 1–4 together need about 30–40 minutes of T4 time.

1. **Repair the metric** (about an hour of code, no GPU).

- Score fact tokens only, such as the log-probability of "Calderfen", not the whole sentence.
- Add post-attack truth ratio and post-attack retain log-prob to every durability measurement.
- Add fact accuracy under greedy generation, before and after attack.
- Plot durability against retain as a frontier. A method wins only if it sits below the curve the others trace.

2. **Port the real HRU mechanism (v5).** Bring `_AttackHardener` into the Colab pipeline, upgraded from batch replay to per-example weights: each example's loss drop on the attacked clone sets its weight in the next NPO step.

3. **Run an equal-budget comparison** over three seeds, no sharding, 120 updates each.

Arm	Isolates
NPO-120	Baseline at matched budget
GA-120	Damage-only control
SAM-NPO-120	Current "HRU" without the mechanism
HRU-v5-120	SAM-NPO plus attack-then-reweight

HRU-v5 against SAM-NPO on post-attack fact recovery, with a paired test across seeds, proves or refutes the central claim.

4. **Run notebook 04b.** It needs no training and shows whether veto aggregation rescues sharding.
5. **Then optimise cost.** Apply v4's techniques to v5. The July hardener is light (3 attack steps on 4 examples every 5 steps), so v5 should land near 2× a scrub pass.
6. **Consolidate to one codebase,** so the method described is the method that produced the numbers.

10. How to present this

A mid-project review rewards a team that states exactly what it knows. A version of the update that survives a sharp question:

"The harness is complete and runs all twelve variants. Our clearest finding is that SISA-style sharding dilutes approximate unlearning: a scrubbed shard at -4.18 drops to -0.50 inside the ensemble, because three untouched shards outvote it. Auditing our results, we found the durability metric tracks overall model damage ($r = 0.88$), so we are repairing it to score fact tokens directly. We also confirmed that HRU's inline hardening was not in the pipeline that produced these numbers. The next run tests it head-to-head against an equal-budget baseline."

Appendix A. Cost accounting

Quantity	Counts	Used for
Updates	Optimizer steps that change the model	Comparing methods
Steps	Forward/backward passes (SAM = 2 per update)	FLOP estimate
Probe updates	Attack steps on throwaway copies	Method overhead (HRU-v5) or validation (v4)
Seconds	Wall-clock time	Sanity check; varies with Colab load

Appendix B. Files

Item	Location
July harness, with <code>_AttackHardener</code>	<code>Documents/Vanta/03_PROJECTS/active/hru-unlearning/arms/hru.py</code>
Notebooks, data, checkpoints, <code>results.csv</code>	Google Drive <code>MyDrive/HRU_project/</code>
Progress deck (slide 11 needs §2 corrections)	<code>Downloads/Machine_Unlearning_UPDATED.pptx</code>

Appendix C. References

- Bourtole et al., *Machine Unlearning* (SISA), IEEE S&P 2021.
- Guo et al., *Certified Data Removal from Machine Learning Models*, ICML 2020, [arXiv:1911.03030](https://arxiv.org/abs/1911.03030).

- Zhang et al., *Negative Preference Optimization*, [arXiv:2404.05868](https://arxiv.org/abs/2404.05868).
- Li et al., *The WMDP Benchmark (RMU)*, [arXiv:2403.03218](https://arxiv.org/abs/2403.03218).
- Maini et al., *TOFU: A Task of Fictitious Unlearning for LLMs*, [arXiv:2401.06121](https://arxiv.org/abs/2401.06121).
- Shi et al., *MUSE: Machine Unlearning Six-Way Evaluation*, [arXiv:2407.06460](https://arxiv.org/abs/2407.06460).
- *Jogging the Memory of Unlearned Models Through Targeted Relearning Attacks*, [arXiv:2406.13356](https://arxiv.org/abs/2406.13356).
- *Unlearning Isn't Deletion: Investigating Reversibility of Machine Unlearning in LLMs*, [arXiv:2505.16831](https://arxiv.org/abs/2505.16831).
- *Towards LLM Unlearning Resilient to Relearning Attacks: A Sharpness-Aware Minimization Perspective and Beyond*, [arXiv:2502.05374](https://arxiv.org/abs/2502.05374).
- *Improving LLM Unlearning Robustness via Random Perturbations*, [arXiv:2501.19202](https://arxiv.org/abs/2501.19202).
- Foret et al., *Sharpness-Aware Minimization for Efficiently Improving Generalization*, ICLR 2021.
- Shi et al., *Detecting Pretraining Data from Large Language Models (Min-K% Prob)*, [arXiv:2310.16789](https://arxiv.org/abs/2310.16789).
- Chen et al., *When Machine Unlearning Jeopardizes Privacy*, CCS 2021.
- Duan et al., 2024, on membership inference against LLMs, as discussed in [arXiv:2412.13475](https://arxiv.org/abs/2412.13475).